

What is Node.js? Architecture

What is Node.js?

Node.js is an open-source, cross-platform **JavaScript runtime environment** that allows you to run JavaScript code **outside of the web browser** — typically on the **server side**.

- **Built on:** Node.js is built on **Google Chrome's V8 JavaScript engine**, which compiles JavaScript code directly into native machine code for fast execution.
- **Created by:** Ryan Dahl in 2009.
- **Main use:** It's used for building fast, scalable, and efficient network applications, especially web servers and APIs.

Node.js Architecture Explained

Node.js uses a **Single Threaded Event-Driven Architecture**. Unlike traditional web servers that use multiple threads to handle concurrent requests, Node.js handles multiple requests using a **single thread with asynchronous I/O**.

Key Components of Node.js Architecture:

1. Single Threaded Event Loop

- Node.js runs on a **single thread**.
- It doesn't spawn new threads for each request. Instead, it uses **event-driven, non-blocking I/O** to handle concurrent operations.
- Efficient for I/O-bound tasks like reading from files, querying databases, etc.

2. Event Loop

The **heart of Node.js**. It allows Node.js to perform **non-blocking I/O operations** by offloading operations to the system kernel whenever possible.

Event Loop Phases:

1. **Timers** – executes `setTimeout()` and `setInterval()` .
 2. **Pending Callbacks** – executes I/O callbacks deferred to the next loop.
 3. **Idle, Prepare** – used internally.
 4. **Poll** – fetches new I/O events; executes I/O-related callbacks.
 5. **Check** – executes `setImmediate()` callbacks.
 6. **Close Callbacks** – handles close events like `socket.on('close')` .
-

3. Libuv

- A C library that provides Node.js with its **event loop and asynchronous I/O**.
 - Abstracts non-blocking operations across platforms (Windows, macOS, Linux).
-

4. V8 JavaScript Engine

- Developed by Google.
 - Compiles JavaScript to machine code.
 - Powers both Google Chrome and Node.js.
-

5. Callback Queue / Task Queue

- All async functions (e.g., file read, API call) are processed and when completed, their **callbacks are pushed to the queue** to be executed by the event loop.
-

6. Thread Pool (via libuv)

- Used for some blocking operations like file I/O, DNS, compression.
 - Node.js can use a **worker thread pool** (default is 4 threads) behind the scenes.
-

Summary

Feature	Node.js
Language	JavaScript
Runtime	Chrome V8
Architecture	Single-threaded, event-driven, non-blocking
Core Concept	Asynchronous I/O
Best Use	Real-time apps, APIs, microservices, chat apps

Let me know if you'd like a **diagram of the architecture** or an **example to visualize the event loop**.